Open in app

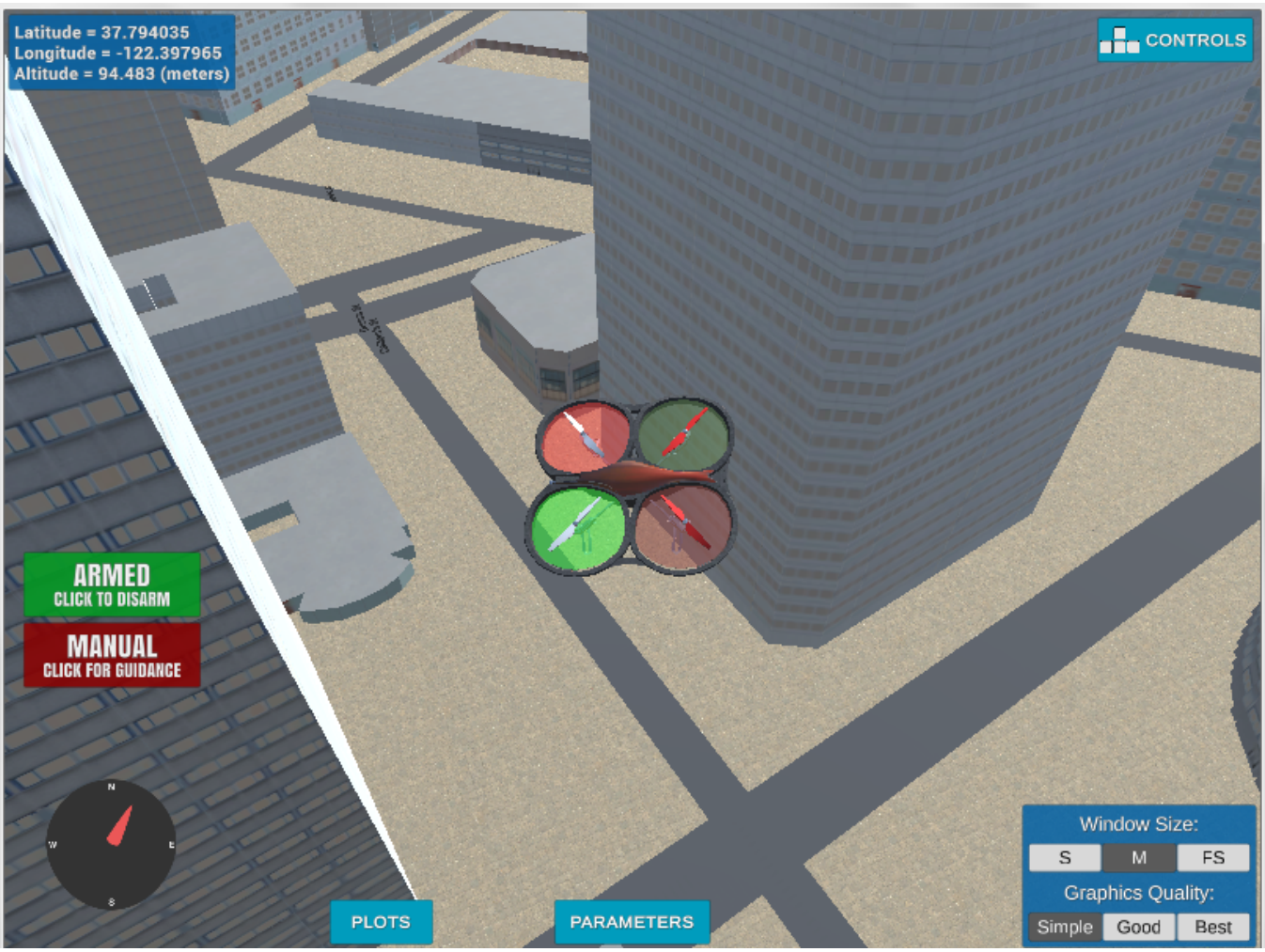
towards
data science

Following 588K Followers

Let’s be A* — Learn and Code a Path Planning algorithm to fly a Drone

In this tutorial we will learn and code a very famous algorithm commonly used for path planning called A* (A — Star)

Percy Jaiswal Apr 30, 2019 · 11 min read



Open in app

from a start location to a goal. Planning is a one of the core capabilities of any autonomous vehicle. Before any autonomous vehicle can embark on a mission, it needs to have a plan, which is nothing but a series of actions that the vehicle must take in order to safely and efficiently move from some initial location to some goal location. To start with a planning problem, we need to have following — a) A set of all possible sets in which a vehicle can find itself (a map of the environment it needs to operate in) b) A start state and a goal state. c) Set of actions which will allow the vehicle to move from one state to next. And finally, d) — Cost of each possible action while transitioning from one state to another.

- **Search space, Action set and Cost**
  So what could be the simplest representation we can think of to represent a "search space"? Let's consider a top-down two-dimensional view of the world. We can divide this world in small grids. Next, we need to incorporate obstacles into this representation. A common way to do this is to mark any grid cell that contains the obstacle as infeasible. The vehicle won't be allowed to enter these cells. All the remaining grid cells are marked as free cells. To prepare a plan will mean to find out a path which will go through sequence of free grid cells from the start state to goal state.

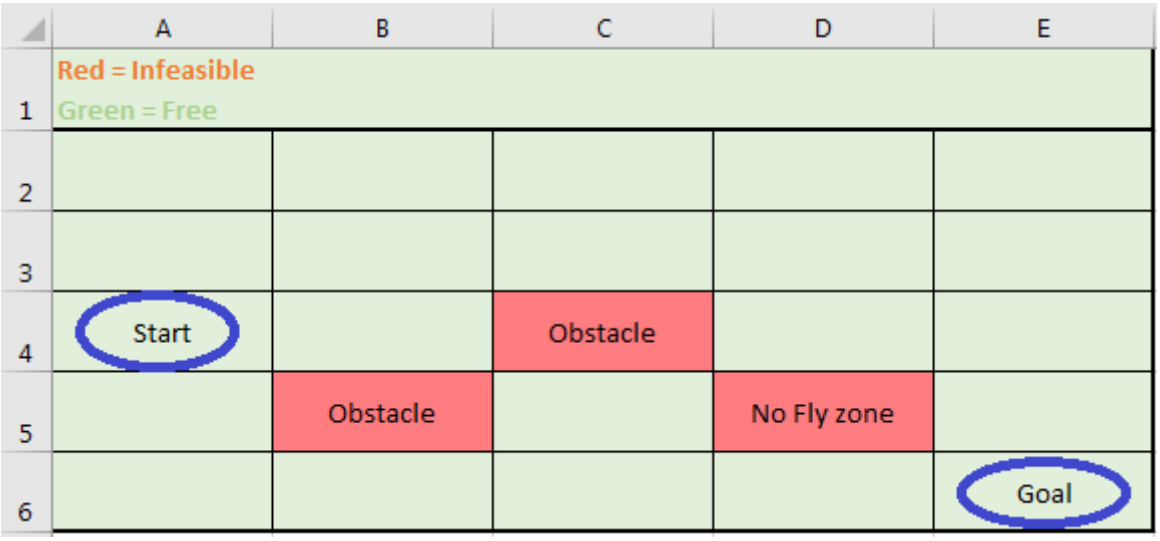


Image 1

Next thing we need is an action set. We can consider actions that move the vehicle right, left, up, down and diagonal motions as our action space. For cost function, we can start simple by considering actions moving *right*, *left*, *up* and *down* as having a cost of 1. For diagonal motion, we can calculate that if lateral and vertical motion costs 1,

Open in app

use that to compare different plans.

- **Breadth First Search (BFS)**
  Now that we have our grid world, we need to build a sequence of steps to travel from start state to goal state. So how do we build this plan? The process of actually searching by 'breadth' can be written down to a few steps, which we'll continue to repeat again and again, until we have reached our goal.
  The backbone of a breadth-first search consists of these basic steps:
  1. Add a node/vertex from the grid to a queue of nodes to be "visited".
  2. Visit the topmost node in the queue, and mark it as such.
  3. If that node has any neighbors, check to see if they have been "visited" or not.
  4. Add any neighboring nodes that still need to be "visited" to the queue.
  5. Remove current node we've visited from the queue.

  Let's see these steps in action by searching for goal location from start position for map shown in above Image 1. We will be maintaining two 'queues' (called 'queue' and 'visited'), one to track nodes which we need to visit, and second one to list down nodes which we have already visited. So starting at 'start' location, let's put that in as first element in our queue. This is step 1 of BFS algorithm, moving to step 2, we will 'visit' 'start' node. The process of visiting a node effectively means that we're noting that it exists, and also checking its neighbouring nodes. In this case, the neighboring nodes of 'start' are nodes A3, B4 and A5; so, we'll add them to our queue. Once we've visited node 'start' and added its neighbors to the queue, we can dequeue node 'start', since we've done everything that we need to do with it. At this moment, our 2 queues will look like this:
  queue -> A3 — B4 — A5; Visited -> Start
  Taking next element from our First In First Out (FIFO) queue, we will visit node A3, check its neighbouring free nodes A2 and B3, add these neighbouring nodes in queue and move A3 from queue to 'visited'. Our updated queues will look like this:
  queue -> B4 — A5 — A2 — B3; Visited -> Start — A3
  Taking one more example, next node in our FIFO queue is B4. Things get a bit interesting here. We see that B4 has nodes B3, 'start', C4 and B5 as its neighbours. We already have identified Obstacles and No Fly Zones in our map, so we will not consider C4 and B5. Moving on we can see 'start' node in our 'visited' list, so we will ignore it. Finally, we see that B3 is already present in queue, so we will ignore this

Open in app

In this case of visiting B4, you can see how BFS algorithm was cleverly able to recognize the fact that 'start' node was previously visited and node B3 was already listed in queue. Updated queues will look like this:
queue -> A5 — A2 — B3; Visited -> Start — A3 — B4
Next we take A5 and repeat the same procedure. And we continue this process till we find our 'goal' location.
One very important feature of BFS algorithm is that it fits very nicely in cases where we want to find shortest distance of any node from our starting node. In fact, many BFS algorithms will keep track of every single node's "parent" node. A parent node is the one who first discovered the node in question. Looking back at the iterations we have done above, let's consider the case when we were 'visiting' A3. When we were at A3, we discovered that A2 and B3 are newly discovered nodes. Now while adding A2 and B3 in our queue, if we just keep a note of the fact that it was A3 which had helped us discovered A2 and B3 nodes, we will have a very convenient way of determining shortest distance to any node from 'start' node. If for example, our 'goal' was at A2, once we discover A2 (when we will be 'visiting' A3), we can note that A3 is its parent node. And then, tracing back through 'visited' list, we can find out that 'start' node would be 'parent' node for A3. And like this, we have found shortest path to A2 from 'start' like A2 <- A3 <- 'start'. Although this might seem like a very simple example, I encourage you to perform this exercise for any node of your interest and find out for yourself that if you keep information about 'parent' node and trace it back through 'visited' list, you will always find the shortest path to that node from 'start'. Needless to say this will hold good if you change your 'start' node to any other location.

- **Heuristic**
  Now that we have a map, our start and goal locations and a way to prepare a plan, we can start to add little bit of intelligence to our algorithm. Right now if you see in above Breadth First Search example, we are visiting nodes on a first come first serve basis. For example, if you prefer to look at right node first, every time we are at any node, the node to it's right will get the priority in FIFO queue and we will always visit this 'right' node before other (left, up, down and diagonal ones), even if other nodes are closer to our goal location. This doesn't seem like a very intelligent system. Given a map of the world we need to traverse, we can make a rough estimate of how far goal location is from all other nodes. There are couple of

Open in app

we are visiting to goal location as a criterial to judge how wise it will be to visit this node.

	A	B	C	D	E
1	Red = Infeasible Green = Free				
2	D = 5.65	D = 5	D = 4.4	D = 4.1	D = 4
3	D = 5	D = 4.2	D = 3.6	D = 3.1	D = 3
4	Start	D = 3.6	Obstacle	D = 2.2	D = 2
5	D = 4.1	Obstacle	D = 2.2	No Fly zone	D = 1
6	D = 4	D = 3	D = 2	D = 1	Goal

Another common method is to take Manhattan distance, which is just the sum of x an y distance remaining to get to the goal.

	A	B	C	D	E
1	Red = Infeasible Green = Free				
2	D = 8	D = 7	D = 6	D = 5	D = 4
3	D = 7	D = 6	D = 5	D = 4	D = 3
4	Start	D = 5	Obstacle	D = 3	D = 2
5	D = 5	Obstacle	D = 3	No Fly zone	D = 1
6	D = 4	D = 3	D = 2	D = 1	Goal

Both these methods ignores obstacles and is going to be an underestimate, but it will tell you whether the node you are planning to visit is taking you towards your goal or going further away. Using this distance, we can prioritize the nodes we want to visit first. In above BFS section, we saw that we were selecting order of neighbouring nodes at random. But now we can use this distance information to 'visit' the node which is at shortest distance from our 'goal'.

Open in app

obstacles and difficulty / cost of tracing different paths and are always an underestimate.

So we got a cost function and a heuristic, let's combine them next!

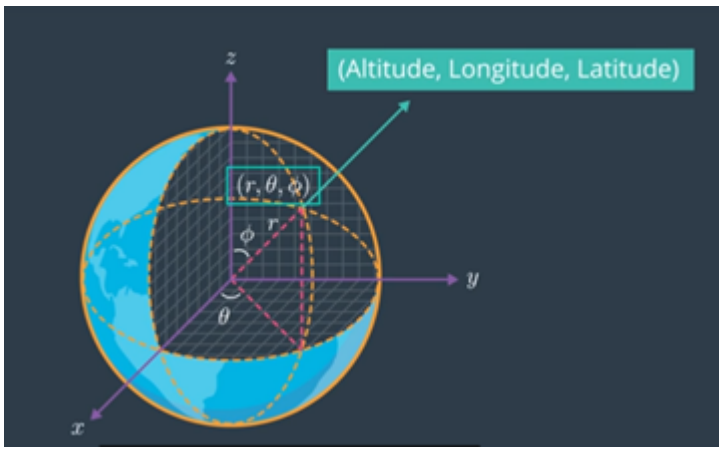
- **A***
  To give guidance to the search process, we have two functions which we can now use. We have the cost function of the sum of the action we have taken so far, let's call it 'G' and we have our heuristic function 'H', which is representation of remaining cost (proportional to remaining distance in our case) to reach goal state. If we add these two quantities together, we get an estimate of the total cost of the plan from the start all the way to the goal, even though we don't know exactly how to get all the way to the goal yet. Let's go back to our example. We will begin at the start state and expand to all neighboring nodes to the right, up and down and we can add a label to each node with the total cost of action 'G'. If you calculate total cost = cost + heuristic for all neighboring nodes, we can see that traveling to right or down has the lowest value of total cost, and that is the direction we will chose to go. This process of choosing a path which has lowest total cost in terms of the actual cost of the actions plus the heuristic from the last node in the plan is the famous algorithm known as **A star.**

	A	B	C	D	E
1	Red = Infeasible Green = Free				
2	D = 8	D = 7	D = 6	D = 5	D = 4
3	D = 7 G = 1	D = 6	D = 5	D = 4	D = 3
4	Start	D = 5 G = 1	Obstacle	D = 3	D = 2
5	D = 5 G = 1	Obstacle	D = 3	No Fly zone	D = 1
6	D = 4	D = 3	D = 2	D = 1	Goal

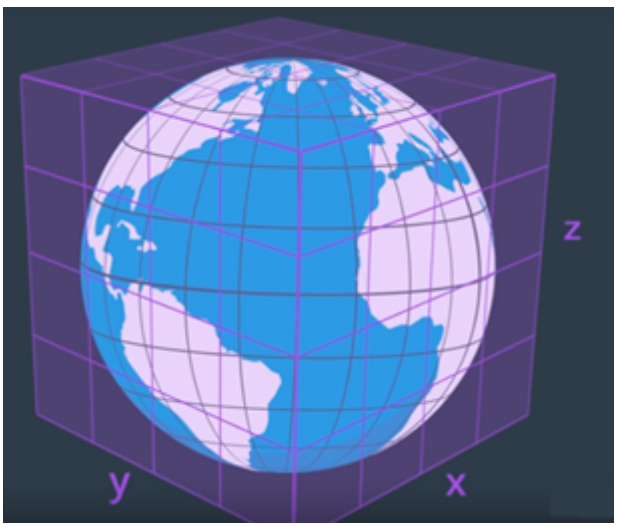
- **Co-ordinate frames**
  Upto this point, we been considering how to find a path from start state to goal state by connecting cells in a grid. But before finding a path to goal location, we first need to represent current position of our vehicle. We will use two different co-

Open in app

position, which are known as geodetic frame. The geodetic frame is just a spherical coordinate system where the typical coordinates 'r', theta (θ) and phi (φ) are represented by altitude, longitude and latitude respectively. Instead of having 'r' = altitude to be 0 at centre, we set it to be 0 at surface of the earth.



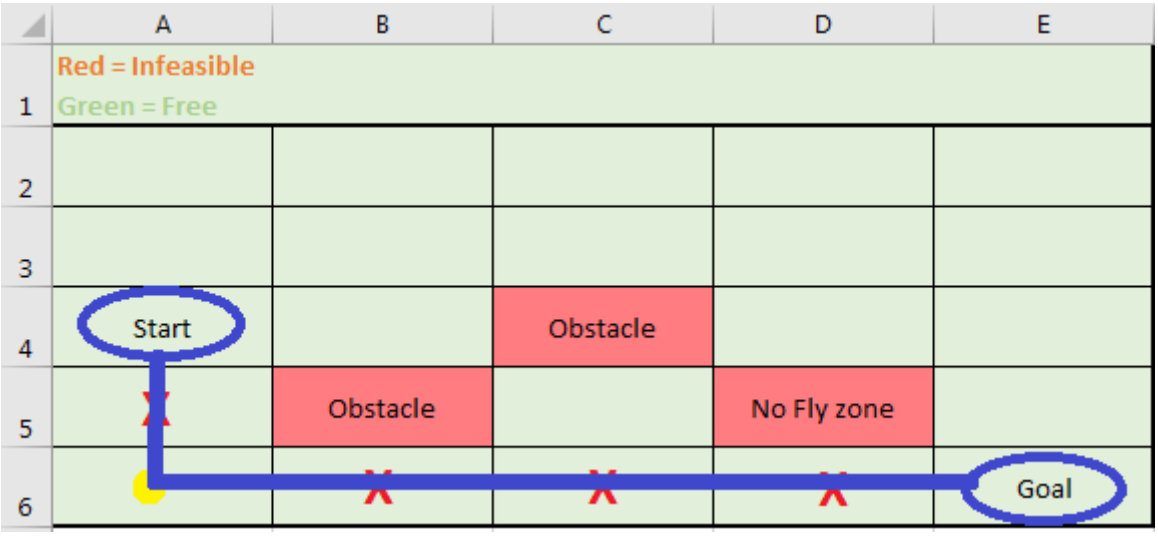
A spherical system can be little difficult to actually do motion planning because its hard to compute distances using angular quantities. Hence we will convert them to more convenient local frame, called as Earth-centred Earth fixed (ECEF) frame. In ECEF, every point in space is represented as x, y and z. However, its more useful if this coordinate system has origin at surface of the earth. And with that we get our own local ECEF frame.



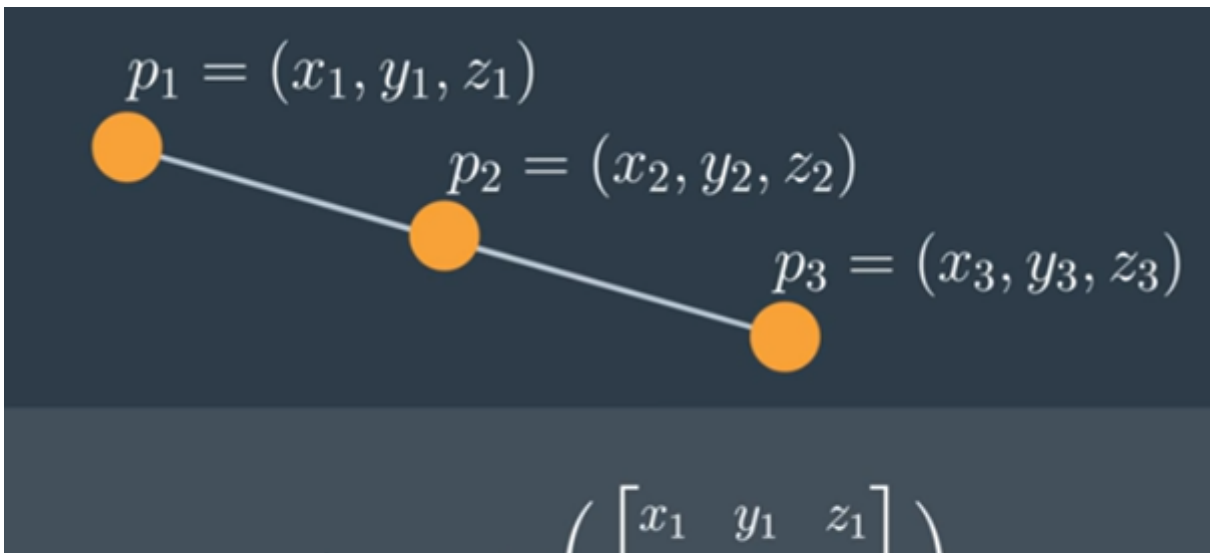
- **Collinearity Check**
  Now that we have a plan output from our A* algorithm, we know how to reach goal location from our starting point. But if we send this plan, the sequence of grid cells to autopilot, what we are really asking the vehicle to do is to travel through a

Open in app

it will start from current location, go one grid down and stop. Next we send next waypoint, which is to go one grid cell down again and stop. Next we send waypoint to go one grid cell right and stop and so and so forth. What we really want our vehicle to do is to go down, turn and travel all the way to the last grid cell which is our goal location.



The question then is how do we take this sequence of grid cells and turn it into waypoints? One method we can use is to take original list of grid cells and just take grid cells that are at beginning and end cell of any sequence of states that lie along a straight line. Let's assume we have 3 points as shown below. If these 3 points line on a same line (they are collinear), then the area of the triangle defined by these three points is zero. If we point coordinates of these 3 points in a matrix form as shown below, then determinant of this matrix represents the area formed by these 3 points. If determinant of this matrix is 0, the area of triangle is 0 and these 3 points are collinear.



Open in app

In next part of this blog, we will be taking the map of city of San Francisco along with methods we have learnt in this part and going to combine them to code a 3D motion planner algorithm. Stay tuned, If it sounds exciting to you!

Till next time….cheers!!

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

Get this newsletter

Emails will be sent to dhiraj.patra@gmail.com.
Not you?

Autonomous Vehicles | Path Planning | Drones | Artificial Intelligence | Self Driving Cars

Medium

About Write Help Legal

Get the Medium app

Download on the App Store

GET IT ON Google Play